Salekh Mamadaliev

M25-SNE-01

Network Engineering

IPv4 & IPv6

Task 1 - Ports and Protocols

1. Using topology from previous lab, replaced Worker VPCS with Worker Ubuntu Cloud Guest:

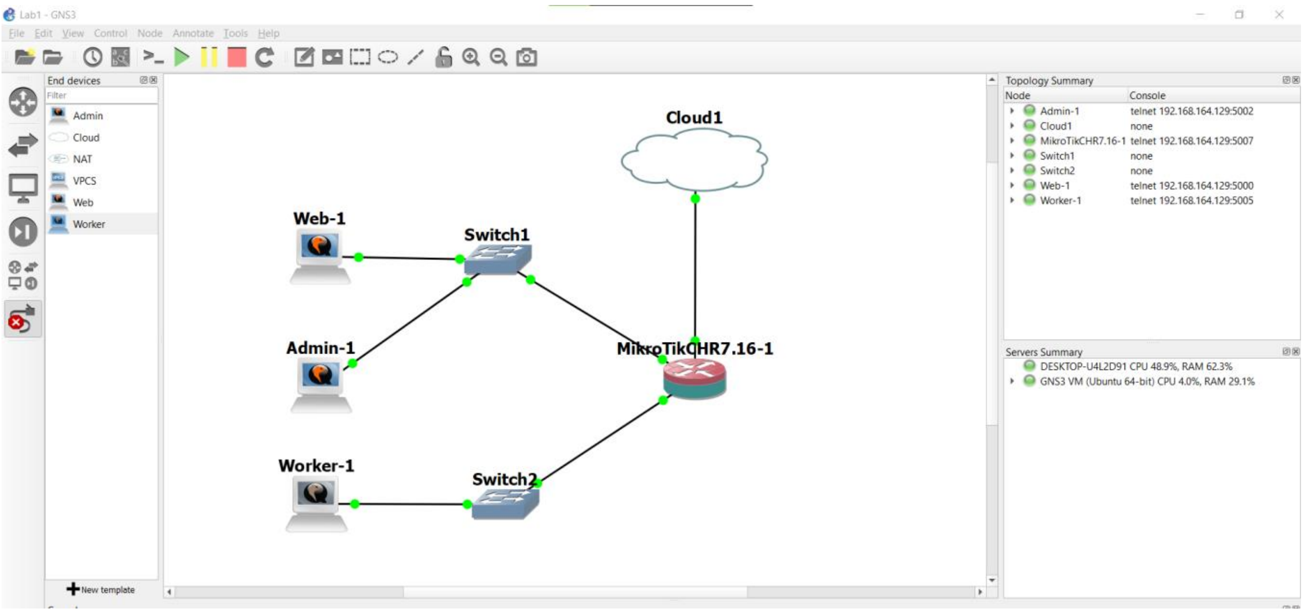


Figure 1 Topology for Task 1

Checking open ports on Web and Admin machines:

```
ubuntu@ubuntu-cloud:~$ sudo netstat -at
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 localhost:domain        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:http            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN
tcp6       0      0 [::]:http               [::]:*                  LISTEN
tcp6       0      0 [::]:ssh                [::]:*                  LISTEN
ubuntu@ubuntu-cloud:~$ sudo lsof -i TCP:80
COMMAND PID     USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
nginx   619     root    6u  IPv4  17919      0t0  TCP *:http (LISTEN)
nginx   619     root    7u  IPv6  17920      0t0  TCP *:http (LISTEN)
nginx   620 www-data    6u  IPv4  17919      0t0  TCP *:http (LISTEN)
nginx   620 www-data    7u  IPv6  17920      0t0  TCP *:http (LISTEN)
ubuntu@ubuntu-cloud:~$
```

```
ubuntu@ubuntu-cloud:~$ sudo ss -tulpn | grep 80
tcp   LISTEN 0      511          0.0.0.0:80        0.0.0.0:*    users:(("nginx",pid=588,fd=6),("nginx",
pid=587,fd=6))
tcp   LISTEN 0      511             [::]:80           [::]:*    users:(("nginx",pid=588,fd=7),("nginx",
pid=587,fd=7))
ubuntu@ubuntu-cloud:~$
```

Figure 2 Checking open ports on Web with lsof, netstat and ss

```
ubuntu@ubuntu-cloud:~$ sudo netstat -at
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 localhost:domain        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN
^C
ubuntu@ubuntu-cloud:~$ sudo lsof -i :22
COMMAND PID USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
sshd    587 root    3u  IPv4  17817      0t0  TCP *:ssh (LISTEN)
sshd    587 root    4u  IPv6  17828      0t0  TCP *:ssh (LISTEN)
ubuntu@ubuntu-cloud:~$
```

```
ubuntu@ubuntu-cloud:~$ sudo ss -tulpn | grep 22
tcp   LISTEN 0      128          0.0.0.0:22        0.0.0.0:*    users:(("sshd",pid=580,fd=3))

tcp   LISTEN 0      128             [::]:22           [::]:*    users:(("sshd",pid=580,fd=4))

ubuntu@ubuntu-cloud:~$
```

Figure 3 Checking open ports on Admin with lsof, netstat and ss

NAT options on gateway:

```
[admin@MikroTik] > ip firewall nat print
Flags: X - disabled, I - invalid; D - dynamic
 0    chain=srcnat action=masquerade out-interface=ether3

 1    chain=dstnat action=dst-nat to-addresses=192.168.10.2 to-ports=22 protocol=tcp in-interface=ether
3 dst-port=2222

 2    chain=dstnat action=dst-nat to-addresses=192.168.10.3 to-ports=22 protocol=tcp in-interface=ether
3 dst-port=2223

 3    chain=dstnat action=dst-nat to-addresses=192.168.10.2 to-ports=80 protocol=tcp in-interface=ether
3 dst-port=8080
[admin@MikroTik] >
```

Figure 4 NAT options on gateway

2. Scanning gateway from Ubuntu VM which running gns3 server:

```
salekh@salekhmamadalievm25sne01:~$ nmap -T4 -p- -n 192.168.49.2
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-11-11 08:51 UTC
Nmap scan report for 192.168.49.2
Host is up (0.0052s latency).
Not shown: 65524 closed tcp ports (conn-refused)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
80/tcp   open  http
2000/tcp open  cisco-sccp
2222/tcp open  EtherNetIP-1
2223/tcp open  rockwell-csp2
8080/tcp open  http-proxy
8291/tcp open  unknown
8728/tcp open  unknown
8729/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 27.77 seconds
```

Figure 5 Gateway scan result with enabled servicess

Known open ports are 21-ftp, 22-ssh,23-telnet,80-http.

3. Disabled unnecessary services on gateway:

```
[admin@MikroTik] > ip service print
Flags: X - DISABLED, I - INVALID
Columns: NAME, PORT, CERTIFICATE, VRF, MAX-SESSIONS
#   NAME      PORT  CERTIFICATE  VRF   MAX-SESSIONS
0 X telnet      23               main            20
1 X ftp         21                               20
2 X www         80               main            20
3 X ssh         22               main            20
4 X www-ssl    443  none         main            20
5 X api       8728               main            20
6 X winbox    8291               main            20
7 X api-ssl   8729  none         main            20
```

Figure 6 Disabled services on gateway

Also disabled bandwidth-server on 2000 port:

```
[admin@MikroTik] > tool bandwidth-server print
                  enabled: yes
             authenticate: yes
  allocate-udp-ports-from: 2000
             max-sessions: 100
[admin@MikroTik] > tool bandwidth-server set enabled=no
[admin@MikroTik] > tool bandwidth-server print
                  enabled: no
             authenticate: yes
  allocate-udp-ports-from: 2000
             max-sessions: 100
```

Figure 7 Disable bandwidth-server on gateway

Scanning gateway again, only necessary services are available:

```
salekh@salekhmamadalievm25sne01:~$ nmap -T4 -p- -n 192.168.49.2
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-11-11 12:57 UTC
Nmap scan report for 192.168.49.2
Host is up (0.00088s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT     STATE SERVICE
2222/tcp open  EtherNetIP-1
2223/tcp open  rockwell-csp2
8080/tcp open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 26.83 seconds
```

Figure 8 Gateway scan results with disabled services

4.1 Scanning Worker from admin, in results only port 22-ssh.

Explanation of the nmap options: -T4 option speeds up scan, -p- enables scanning all ports(1-65535), -n disables DNS lookup.

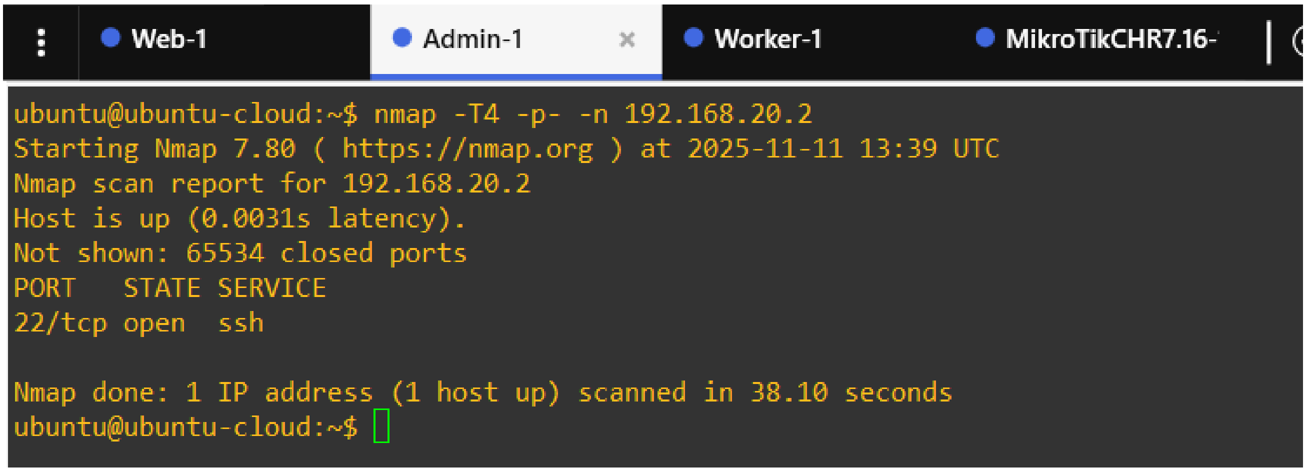


Figure 9 Worker scan results

4.2 Changing SSH port from 22 to 10022:

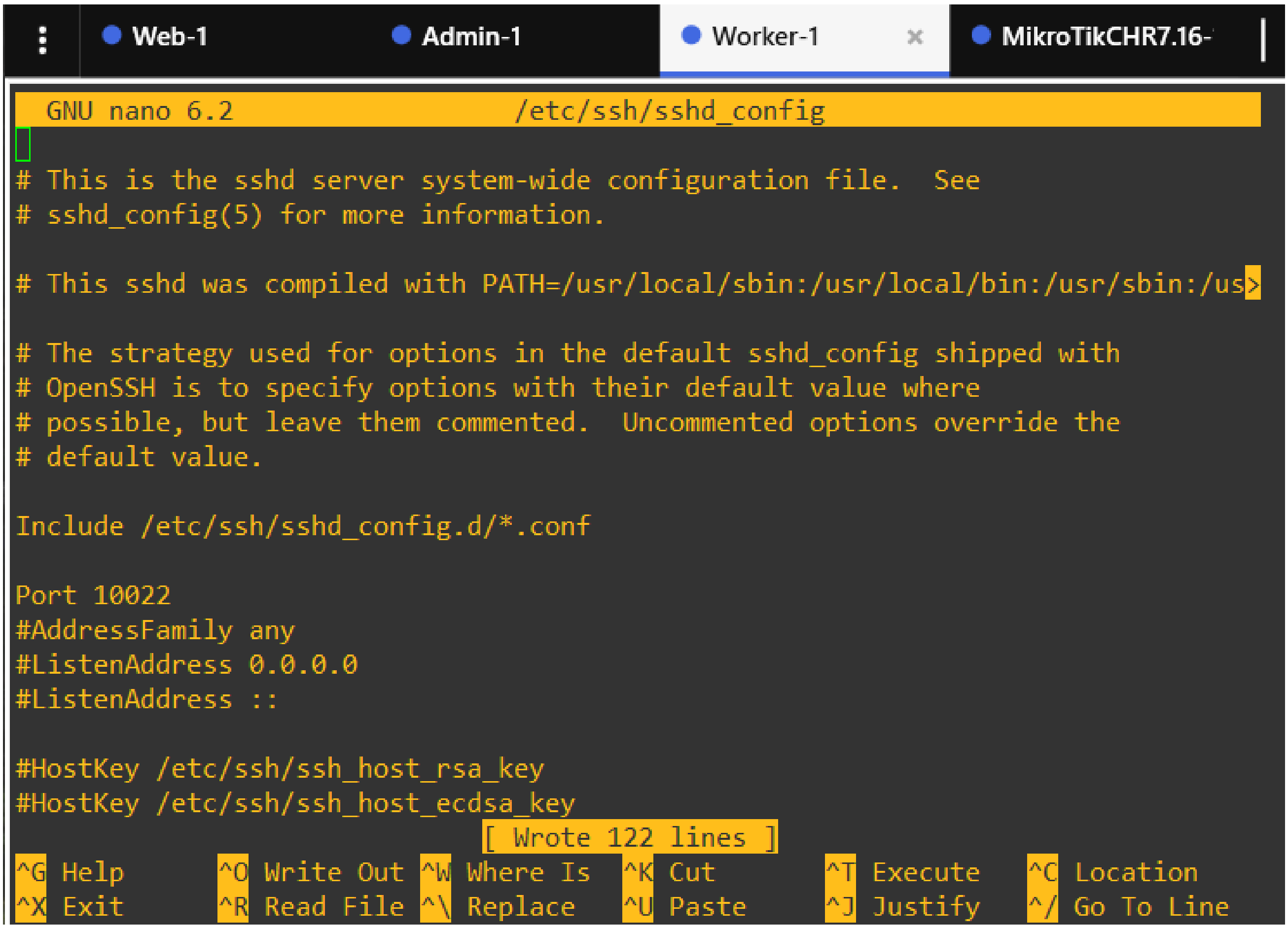


Figure 10 Chaning SSH config

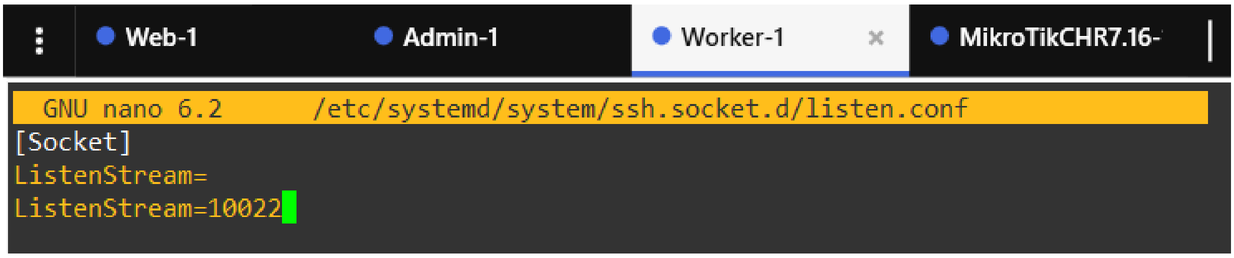
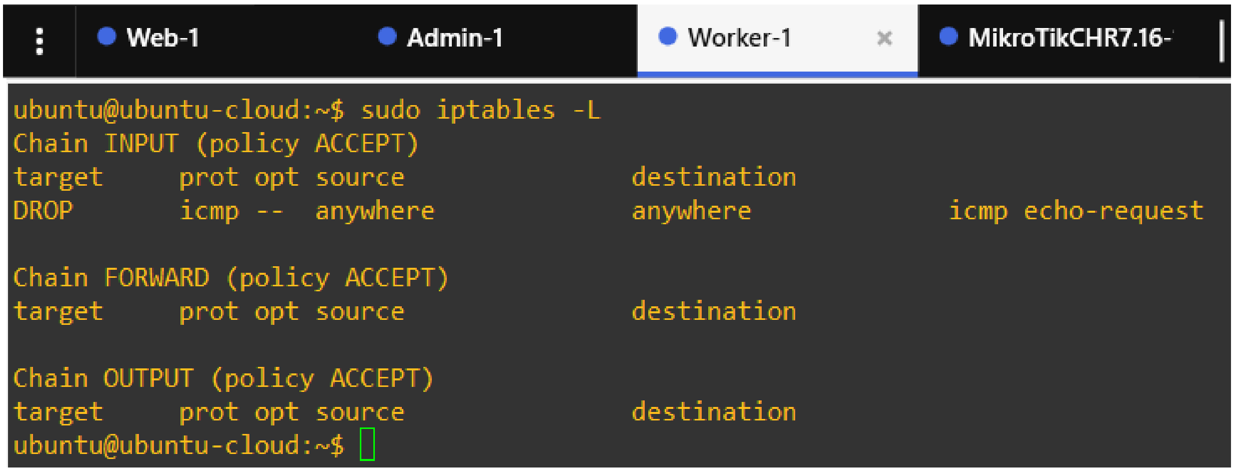


Figure 11 Changing SSH socket config

Restarting services to apply changes and checking results:

Blocking input ICMP traffic on Worker with iptables:



4.3 Scanning Worker without extra options, no open port found:

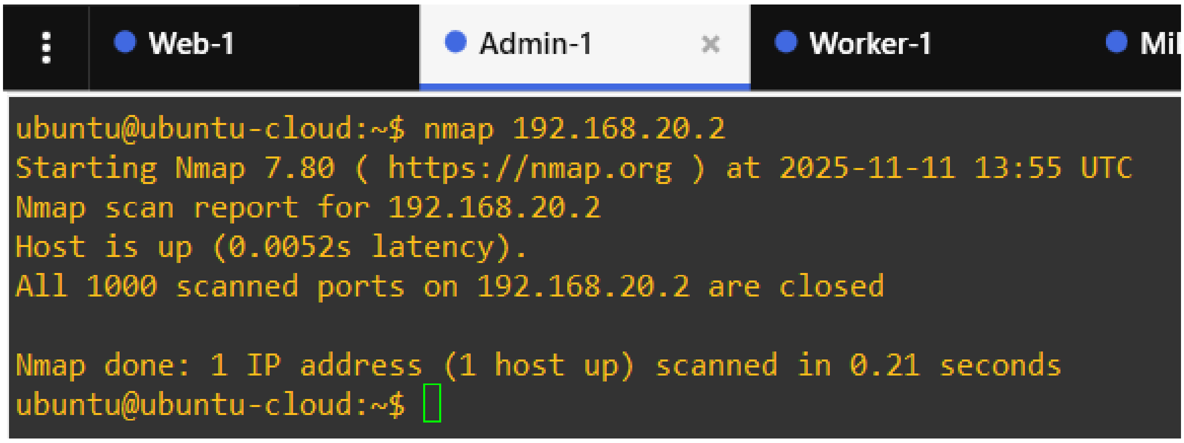


Figure 12 Worker scan without arguments

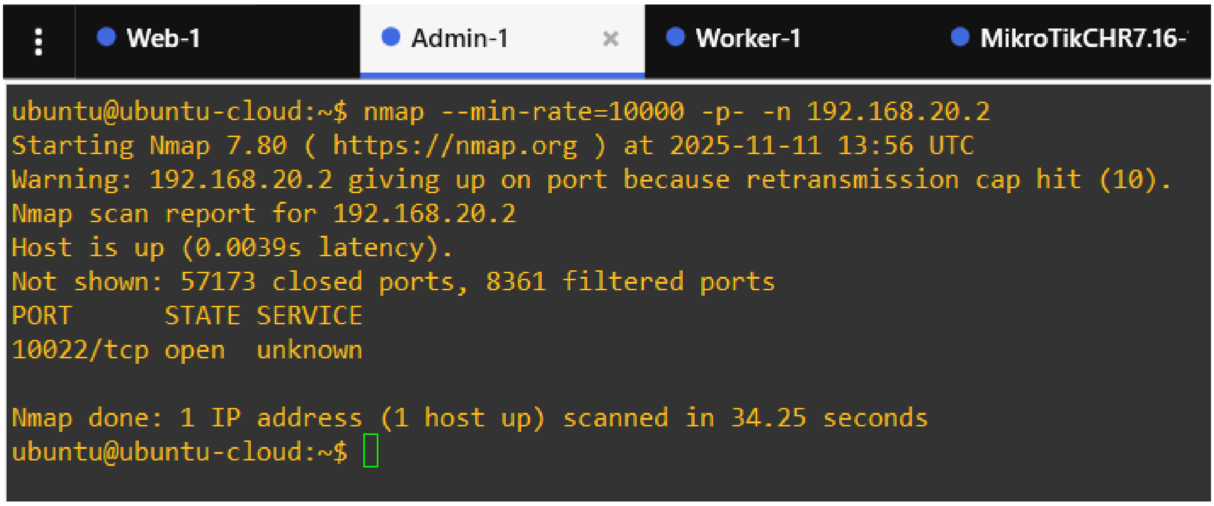
4.4 Scanning Worker again with extra options:

- --min-rate=10000 significantly increases port scan;

- -p- scans all ports;

- -n disables DNS lookup.

Now open SSH port is found.

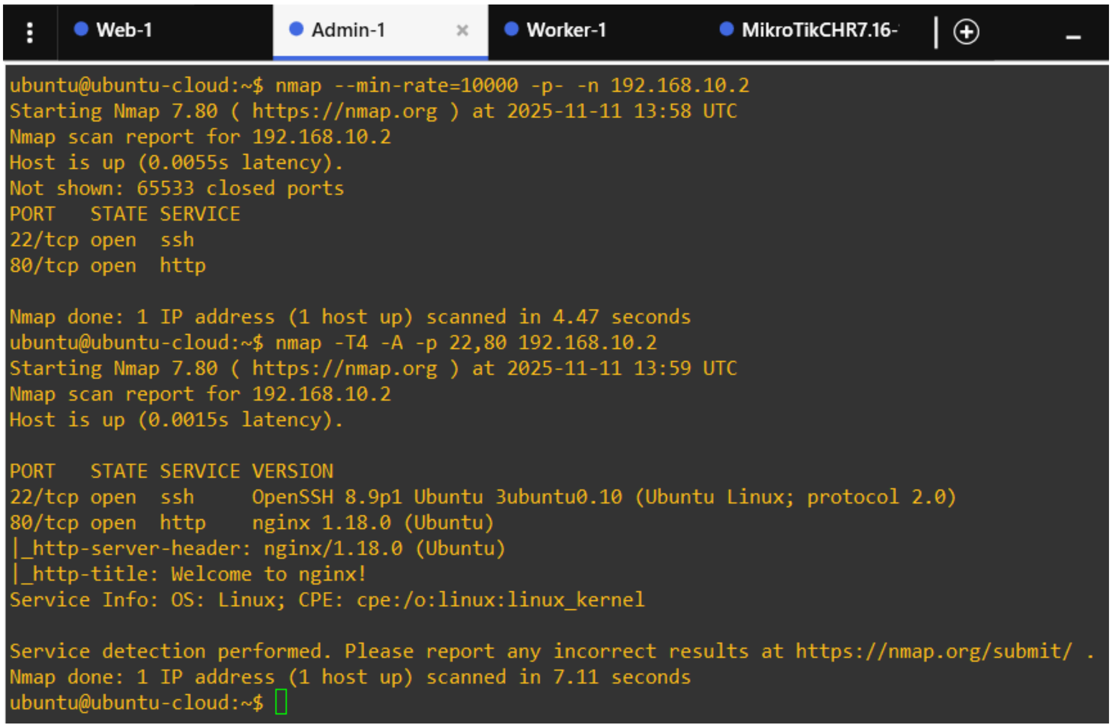



```
ubuntu@ubuntu-cloud:~$ nmap --min-rate=10000 -p- -n 192.168.20.2
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-11 13:56 UTC
Warning: 192.168.20.2 giving up on port because retransmission cap hit (10).
Nmap scan report for 192.168.20.2
Host is up (0.0039s latency).
Not shown: 57173 closed ports, 8361 filtered ports
PORT      STATE SERVICE
10022/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 34.25 seconds
ubuntu@ubuntu-cloud:~$
```

Figure 13 Worker scan with extra options

4.5 To gather more information with nmap, we can perform aggressive scan with option -A, this enables OS detection, version detection, scanning with scripts relative to the open ports, and traceroute.




```
ubuntu@ubuntu-cloud:~$ nmap --min-rate=10000 -p- -n 192.168.10.2
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-11 13:58 UTC
Nmap scan report for 192.168.10.2
Host is up (0.0055s latency).
Not shown: 65533 closed ports
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 4.47 seconds
ubuntu@ubuntu-cloud:~$ nmap -T4 -A -p 22,80 192.168.10.2
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-11 13:59 UTC
Nmap scan report for 192.168.10.2
Host is up (0.0015s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-title: Welcome to nginx!
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.11 seconds
ubuntu@ubuntu-cloud:~$
```

Figure 14 Web aggressive scan results

Task 2 - Traffic Captures

1. Start capturing traffic on the link between Cloud and Gateway:

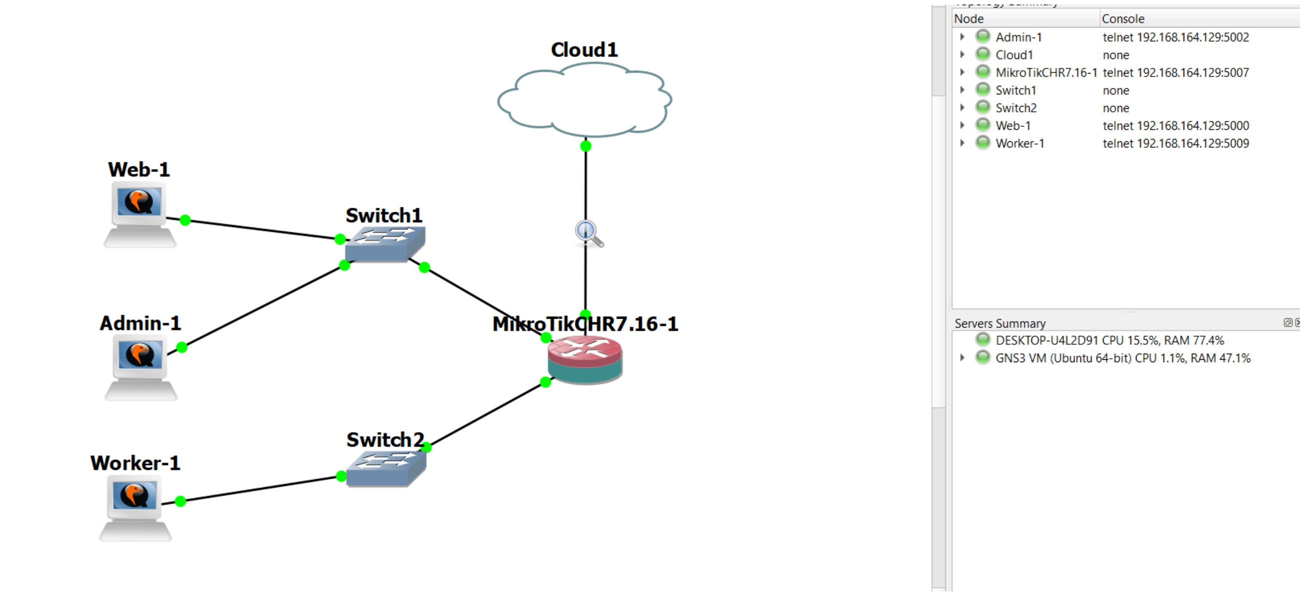
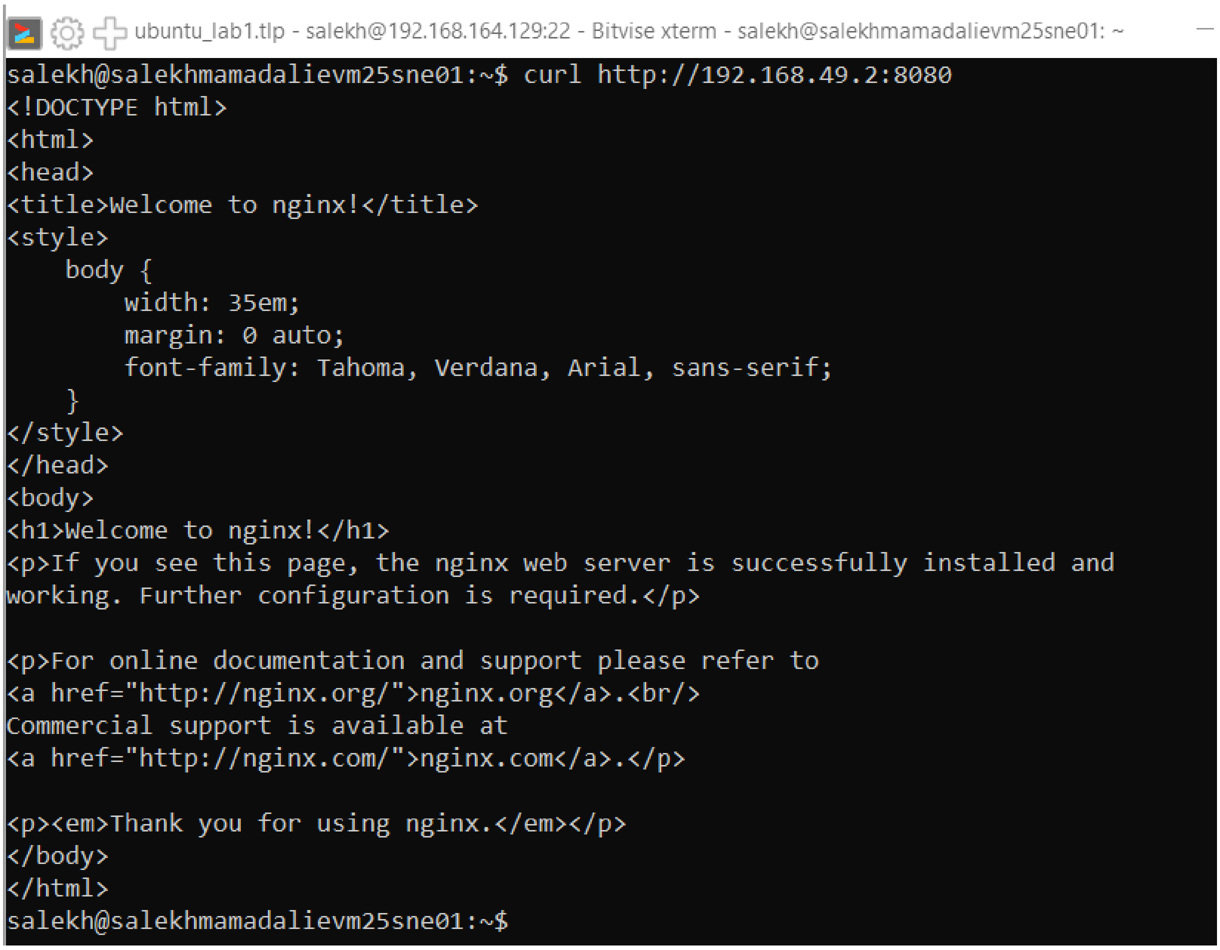


Figure 15 Capturing traffic on Cloud-Gateway link

Requesting web page from Web machine’s http server from Ubuntu VM:



ubuntu_lab1.tlp - salekh@192.168.164.129:22 - Bitvise xterm - salekh@salekhmamadalievm25sne01: ~

```
salekh@salekhmamadalievm25sne01:~$ curl http://192.168.49.2:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
salekh@salekhmamadalievm25sne01:~$
```

Figure 16 Curl request from Ubuntu VM

Exploring captured packets via Wireshark:

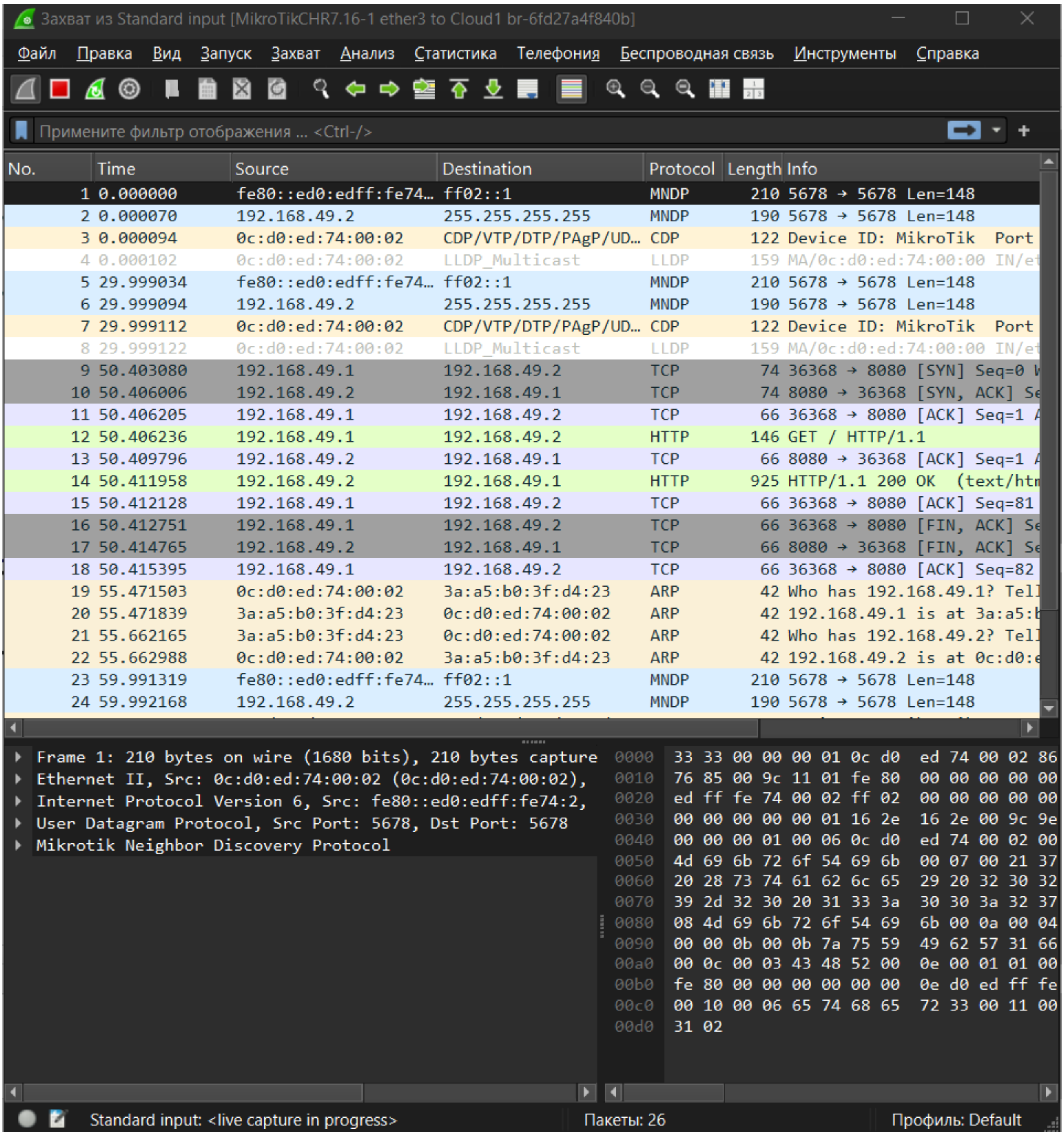


Figure 17 Exploring captured traffic via Wireshark

We can see different packets in this capture: MNDP, CDP, LLDP, TCP, HTTP, ARP. But we are interested in TCP and HTTP protocols packets. On the picture above we can see TCP 3-way handshake: SYN, SYN-ACK, ACK. Then HTTP request from Ubuntu, ACK TCP response and HTTP response from Web server, and ACK from Ubuntu as well. Next three TCP packets show us that the connection has been closed with FIN TCP packets. We can choose any TCP/HTTP packet and follow TCP or HTTP flow to see whole HTTP communication in plaintext:

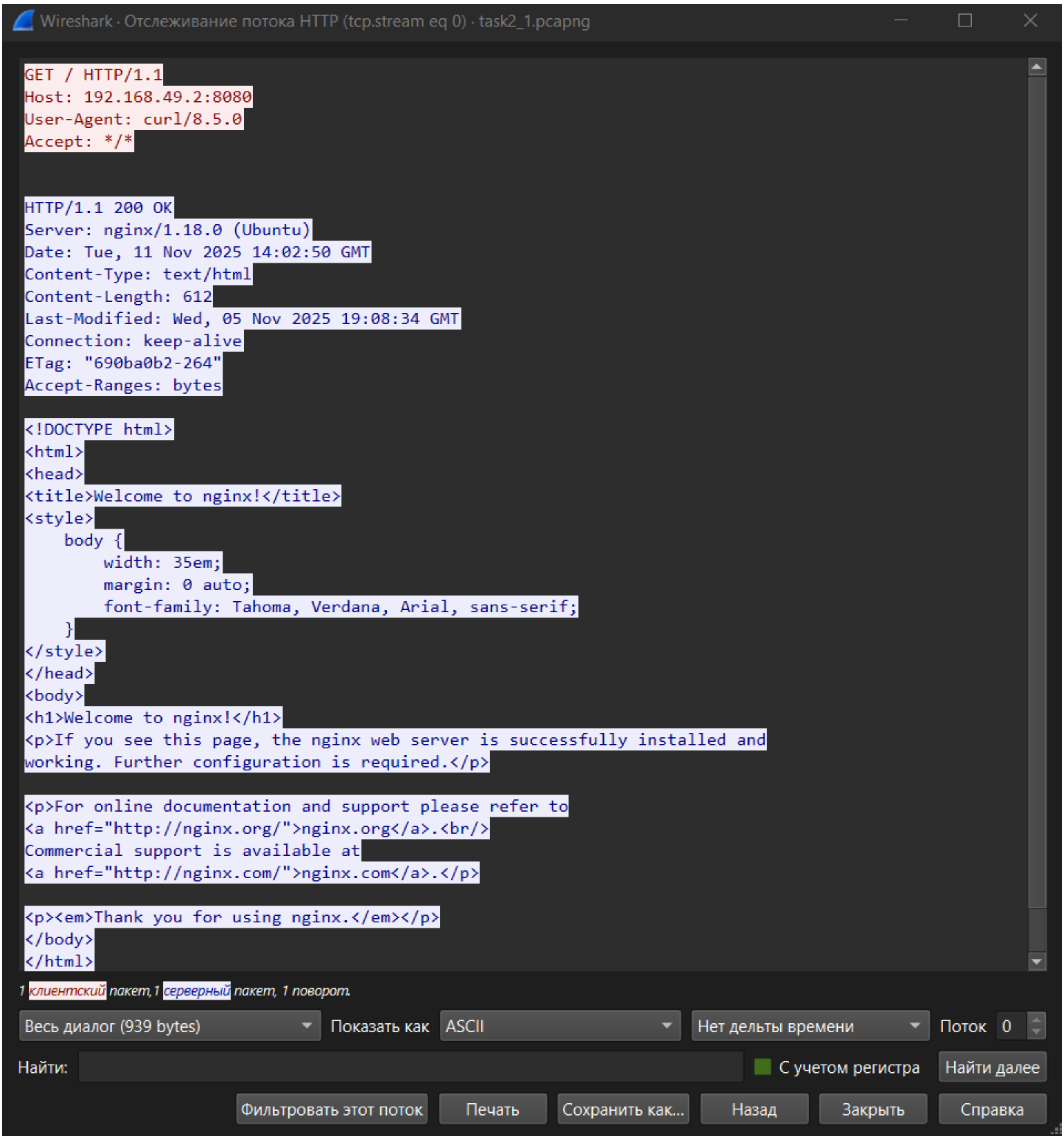


Figure 18 HTTP communication in plaintext

We can see this because there’s no encryption and all HTTP requests/response can be seen to potential attacker.

HTTP headers are metadate fields in request/response, which provide additional information to server about client, and to client about server. They are may have different purposes like storing client session information, define content type of message, or tell server which program is used by client to access server(user-agent header).

2. Connecting to Admin from Ubuntu VM via ssh:

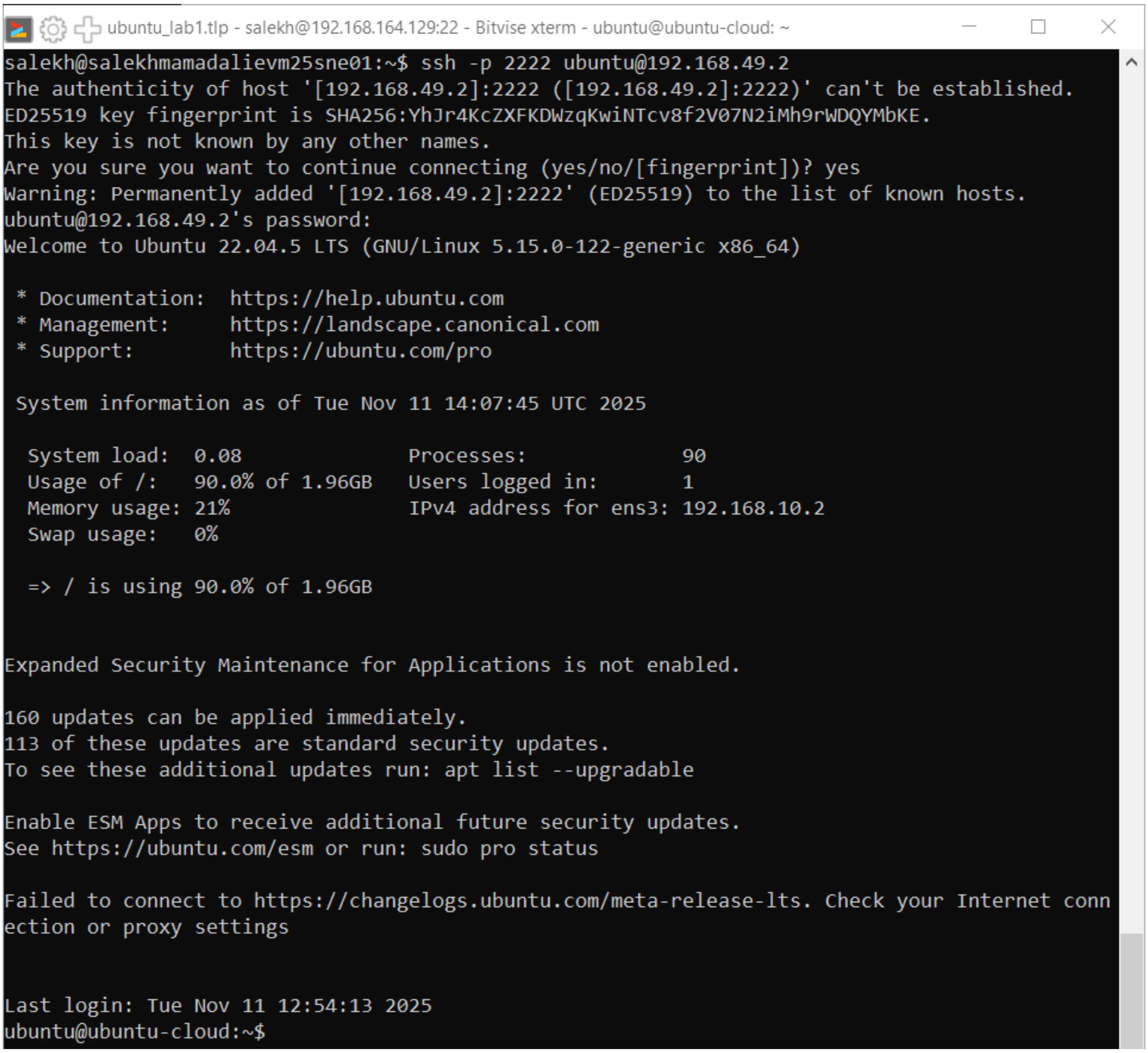


Figure 19 Connecting to Admin via SSH

This time in the capture we have mostly TCP packets.

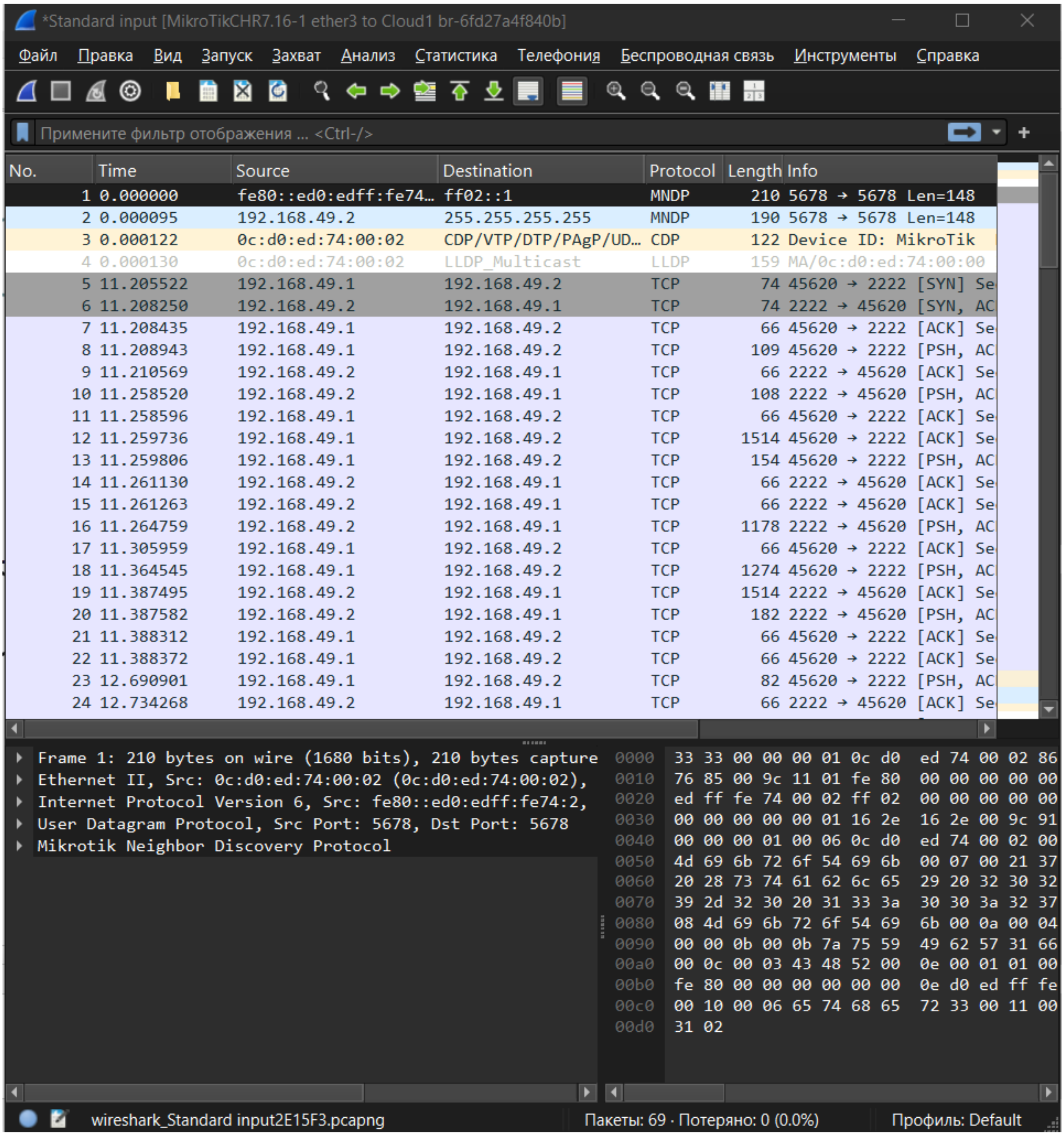


Figure 20 Fragment of captured SSH traffic

TCP flow again starting with 3-way handshake then next packets contain information about SSH version and encryption algorithms. Next packets are encrypted with negotiated algorithm and we cannot see any plaintext data.

Figure 21 TCP flow in SSH connection

We can see that first client and server are exchanging SSH version and supported encryption algorithms. The first matching algorithm in both client's and server's lists will be chosen: in our case that is curve25519-sha256.

3. First. I configured Nginx proxy server on Ubuntu VM to listen at 8080 port and proxy requests to gateway address port 8080. What is because gns3server is installed on Ubuntu VM and GNS user interface is installed on Windows host machine as well as Burp Suite:

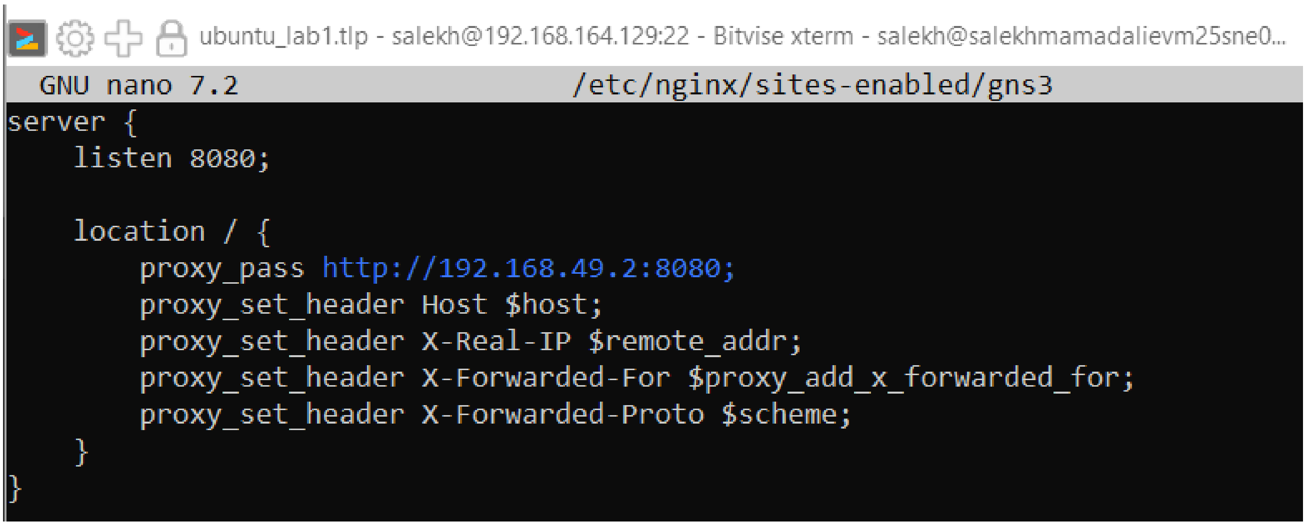


Figure 22 Nginx proxy configuration on Ubuntu VM

Next we connect to Web machine's HTTP server using internal Burp browser, which allow us to intercept HTTP requests in Burp Suite. We intercept simple GET request and can see HTTP request in burp's interface.:

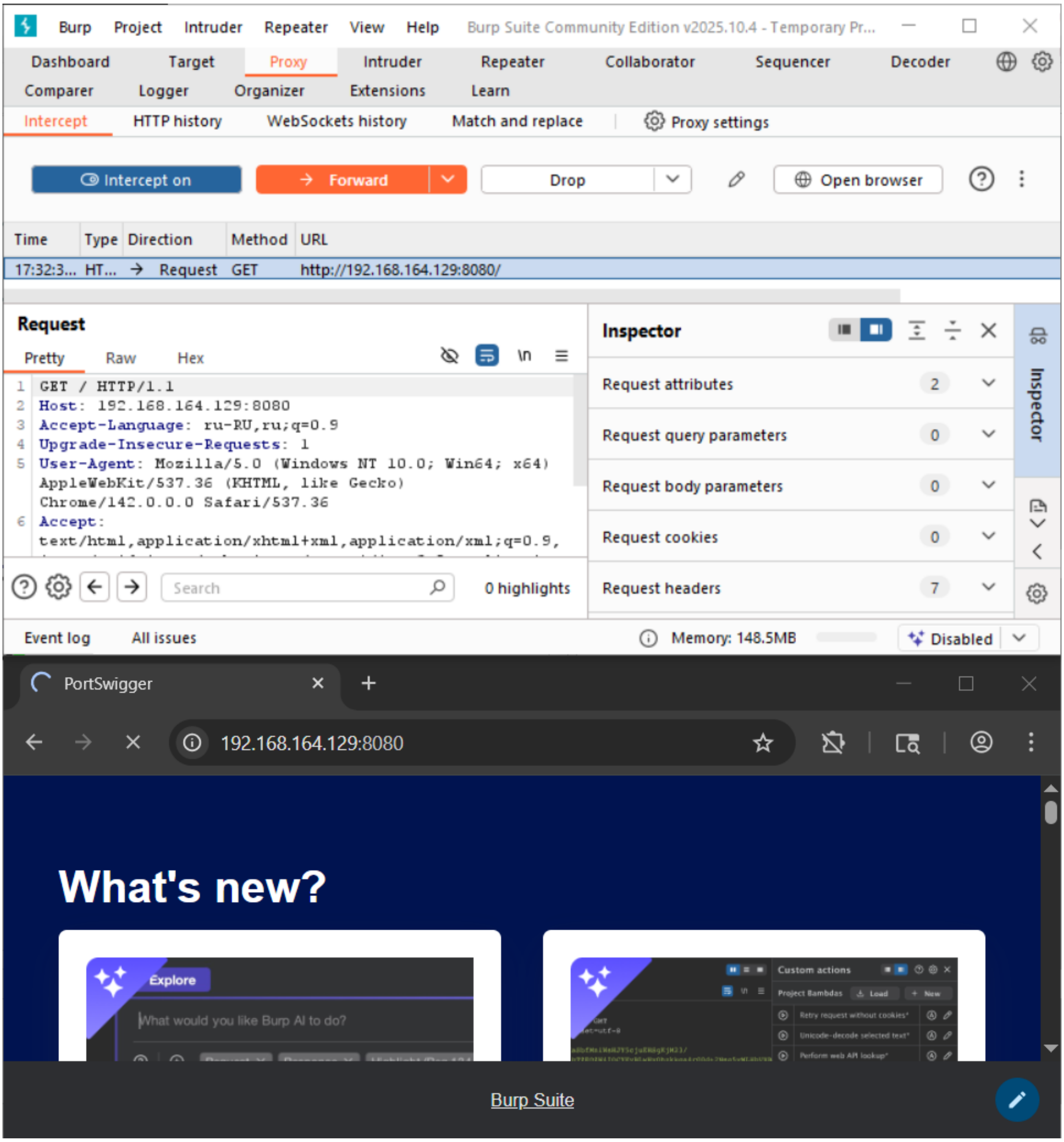


Figure 23 Intercepting HTTP request with Burp Suite

We can modify this HTTP request, for example changing the browser’s user-agent to “New_user_agent”. Then we can forward this request and check the results in captured traffic via Wireshark:

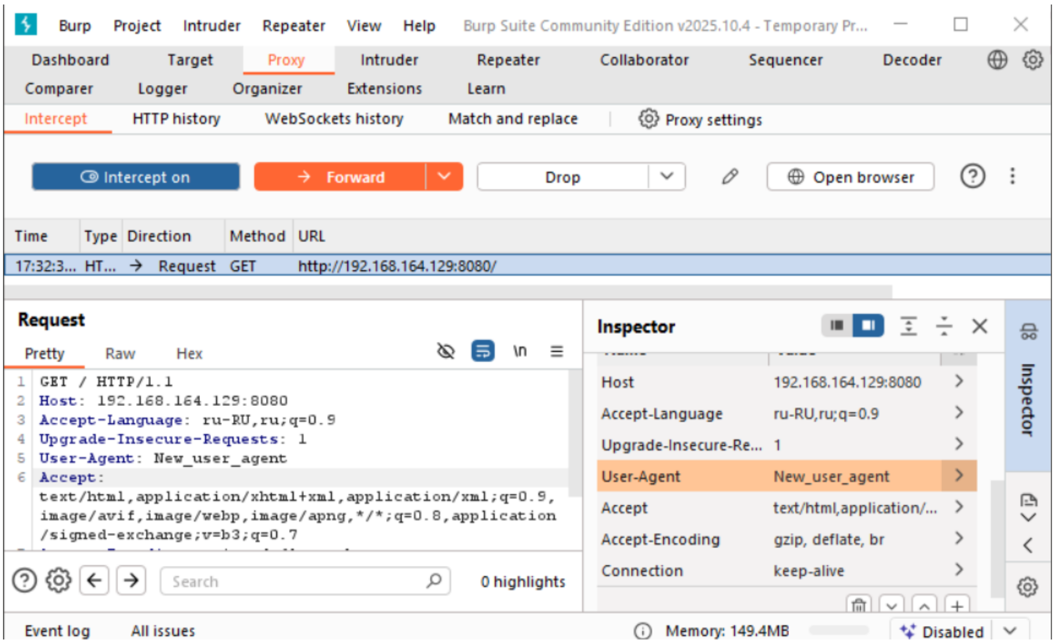


Figure 24 Modifying HTTP user-agent header

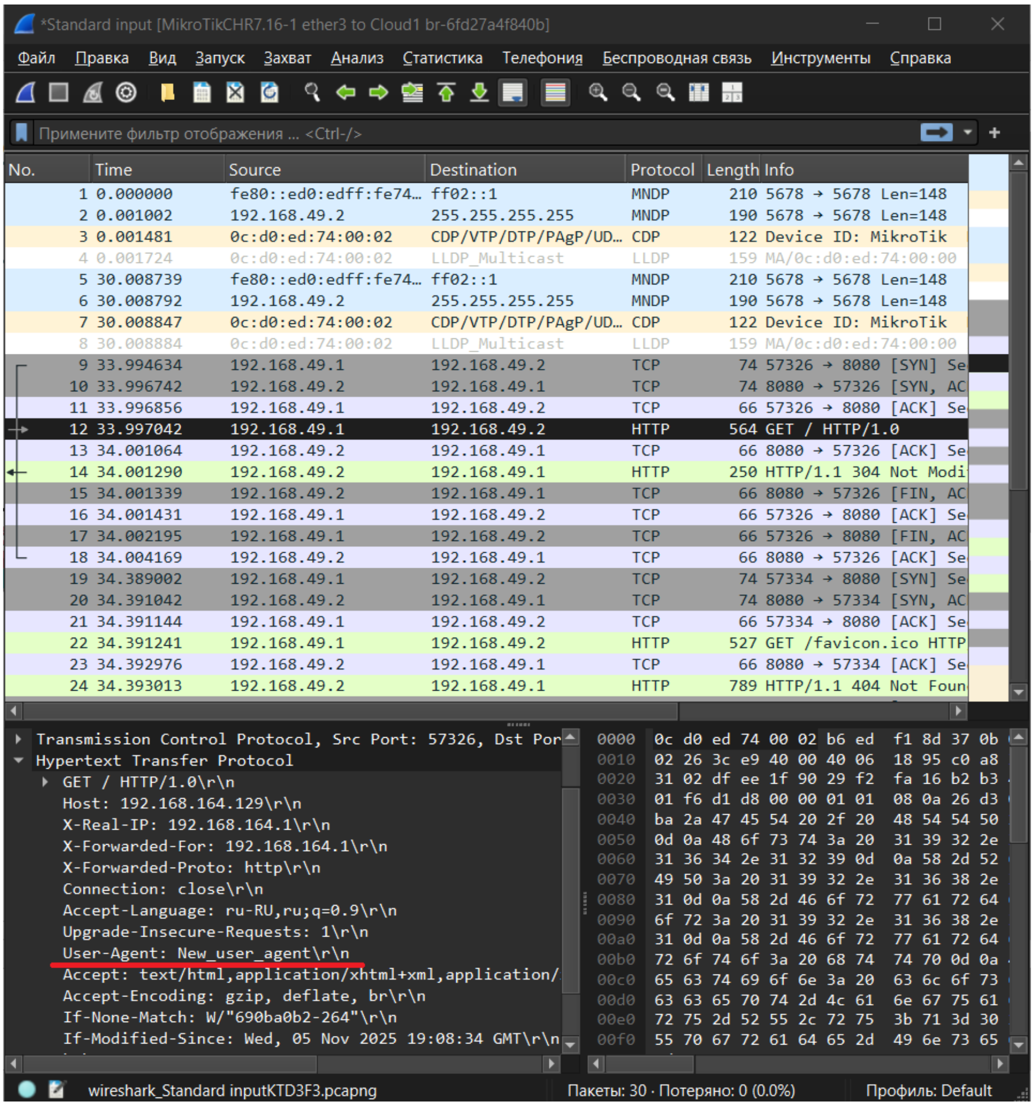


Figure 25 Captured altered HTTP packet

We are able to modify HTTP requests and not SSH requests because there's not encryption in HTTP, so requests/response are transmitted as plaintext. We can use SSL/TLS protocol with HTTP to add secrecy and prevent possible man in the middle attacks.

Task 3 - IPv6

1. Configured IPv6 addresses on each machine:

```
Web-1   Admin-1   Worker-1   MikroTikCHR7.16-

  GNU nano 6.2              /etc/netplan/50-cloud-init.yaml
# /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg with the following:
# network: {config: disabled}
network:
    ethernets:
        ens3:
            addresses:
              - 192.168.10.2/24
              - 2001:db8:1234:1::2/64
            dhcp4: no
            match:
                macaddress: 0c:e8:5d:27:00:00
            set-name: ens3
            routes:
              - to: default
                via: 192.168.10.1
              - to: "::/0"
                via: "2001:db8:1234:1::1"
                on-link: true
    version: 2

^G Help        ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit        ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line
```

Figure 26 Netplan configuration on Web

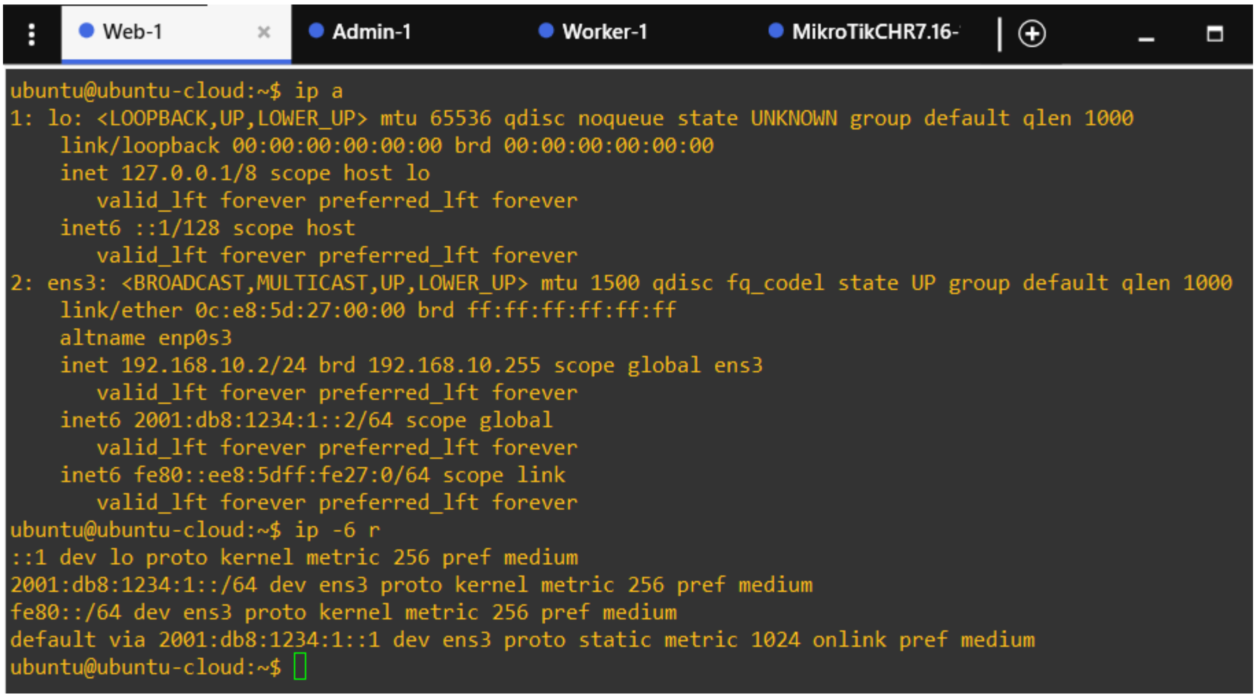


Figure 27 Applied configuration on Web

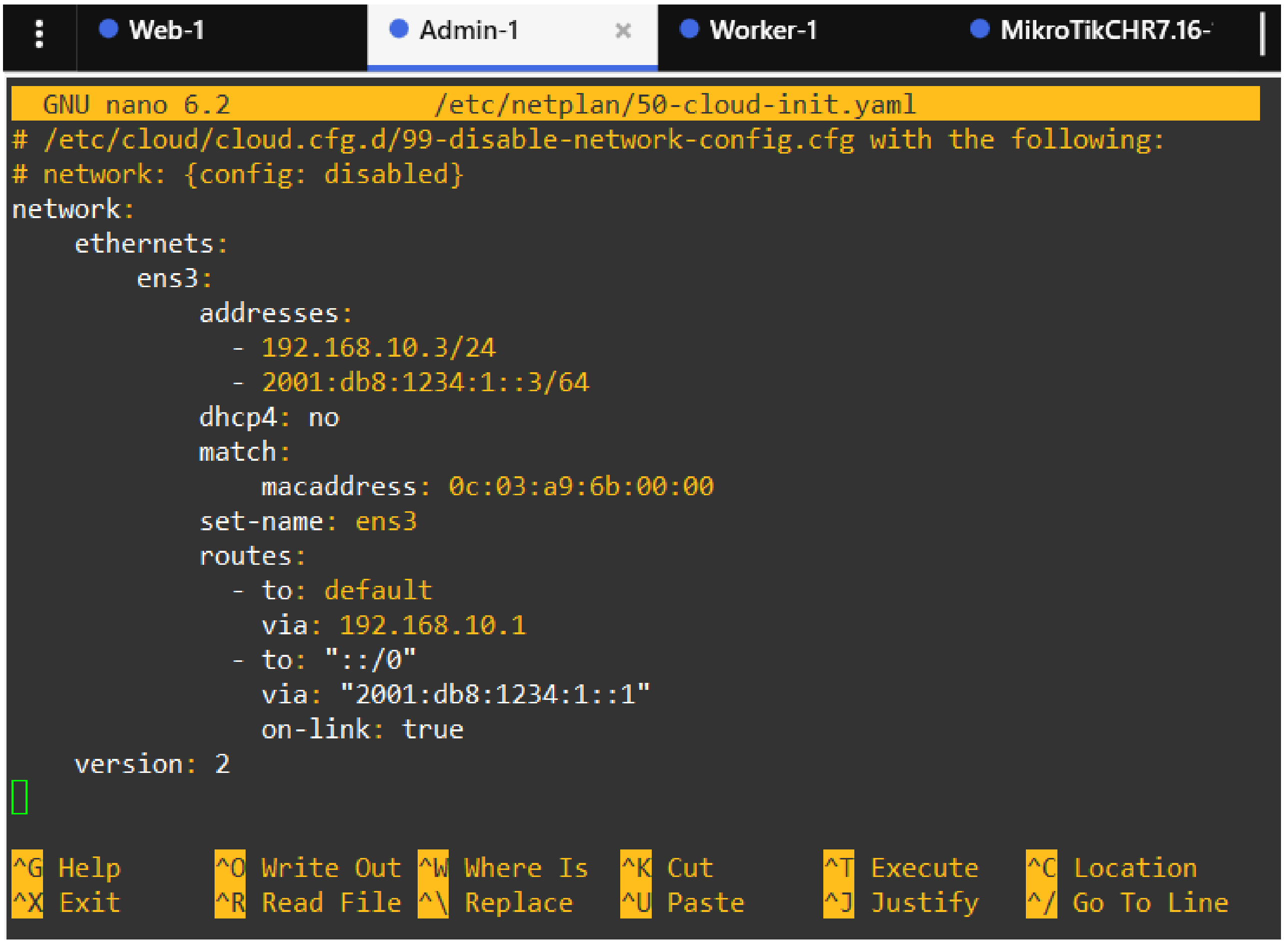


Figure 28 Netplan configuration on Admin

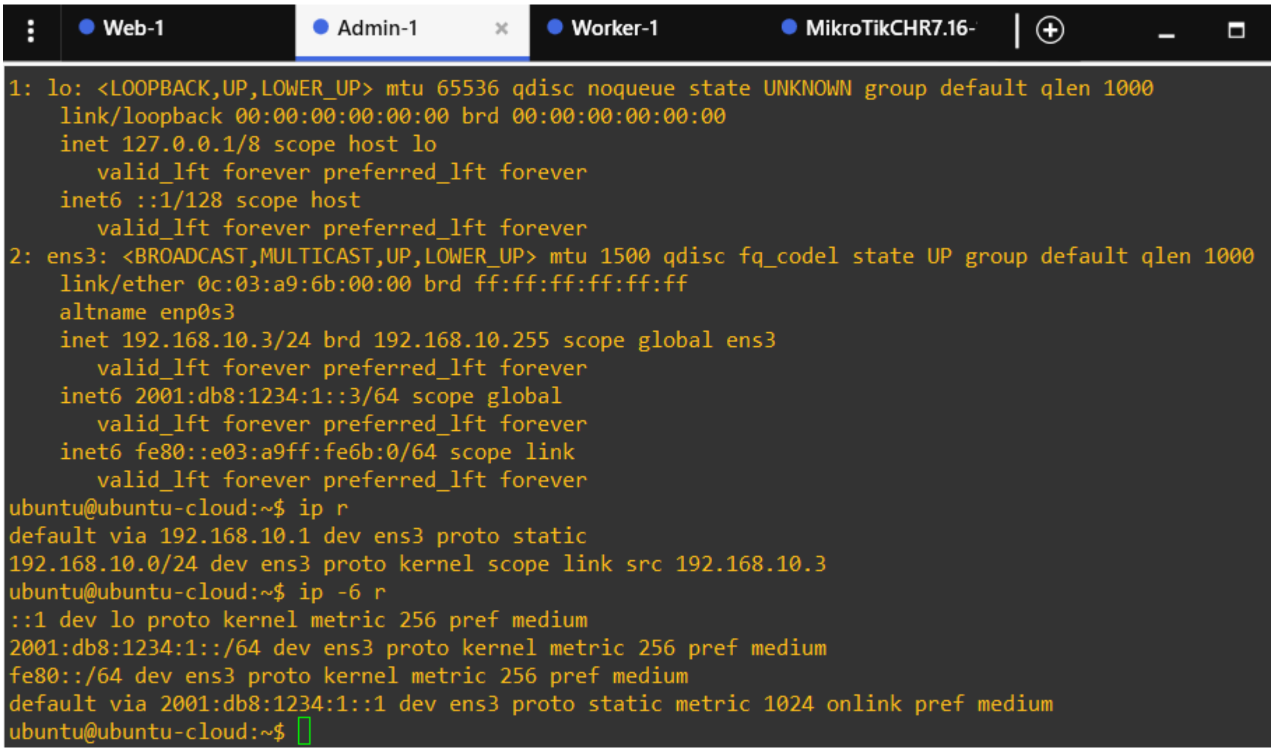


Figure 29 Applied configuration on Admin

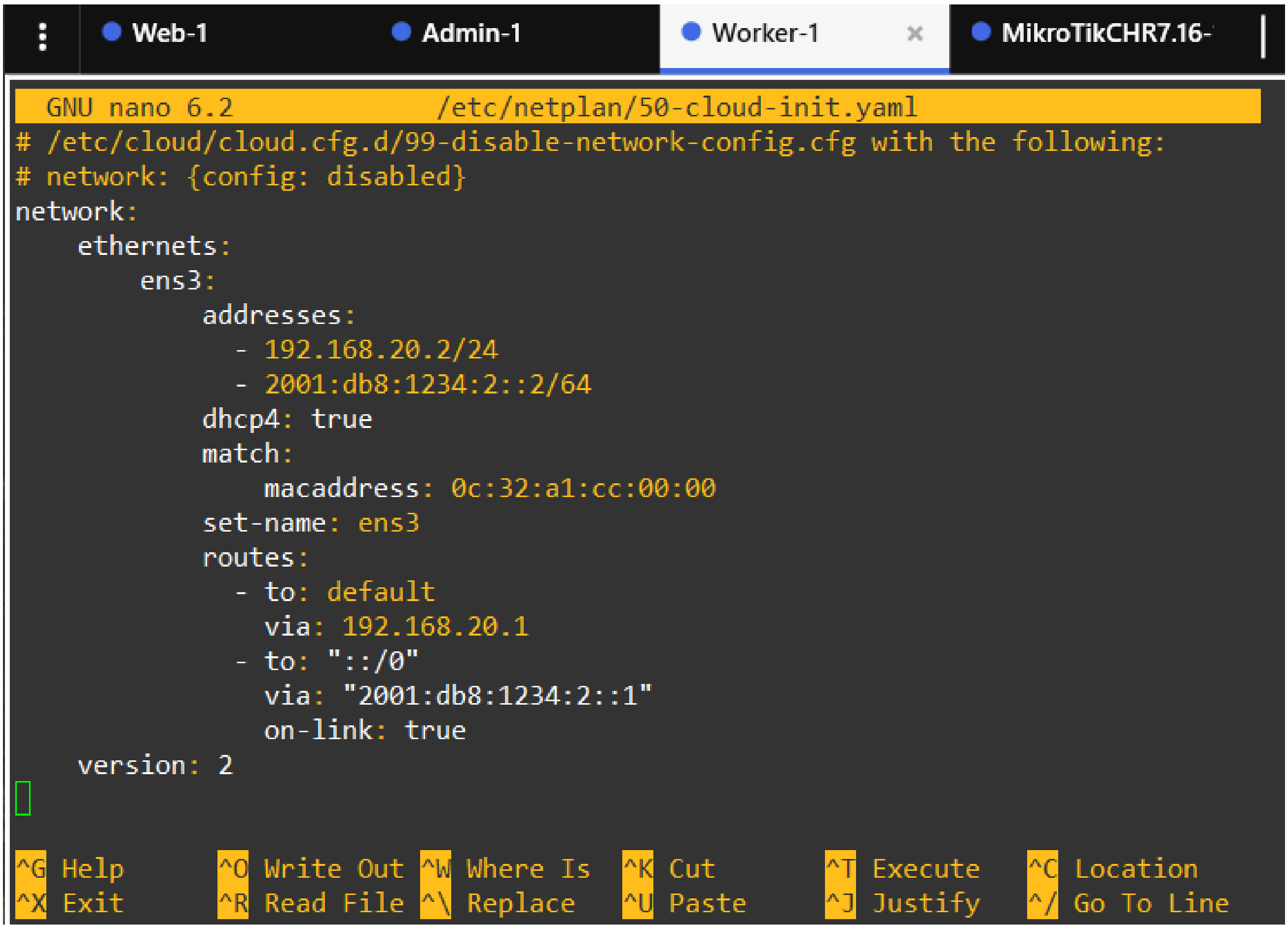
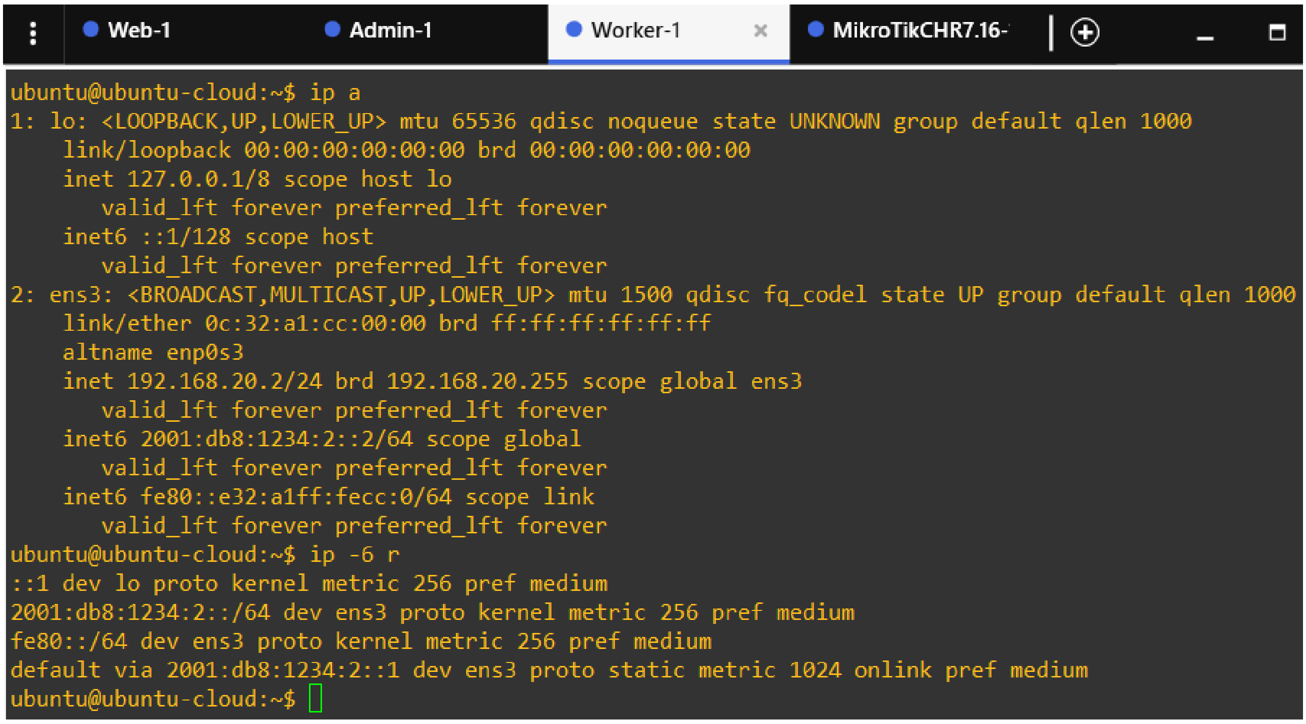
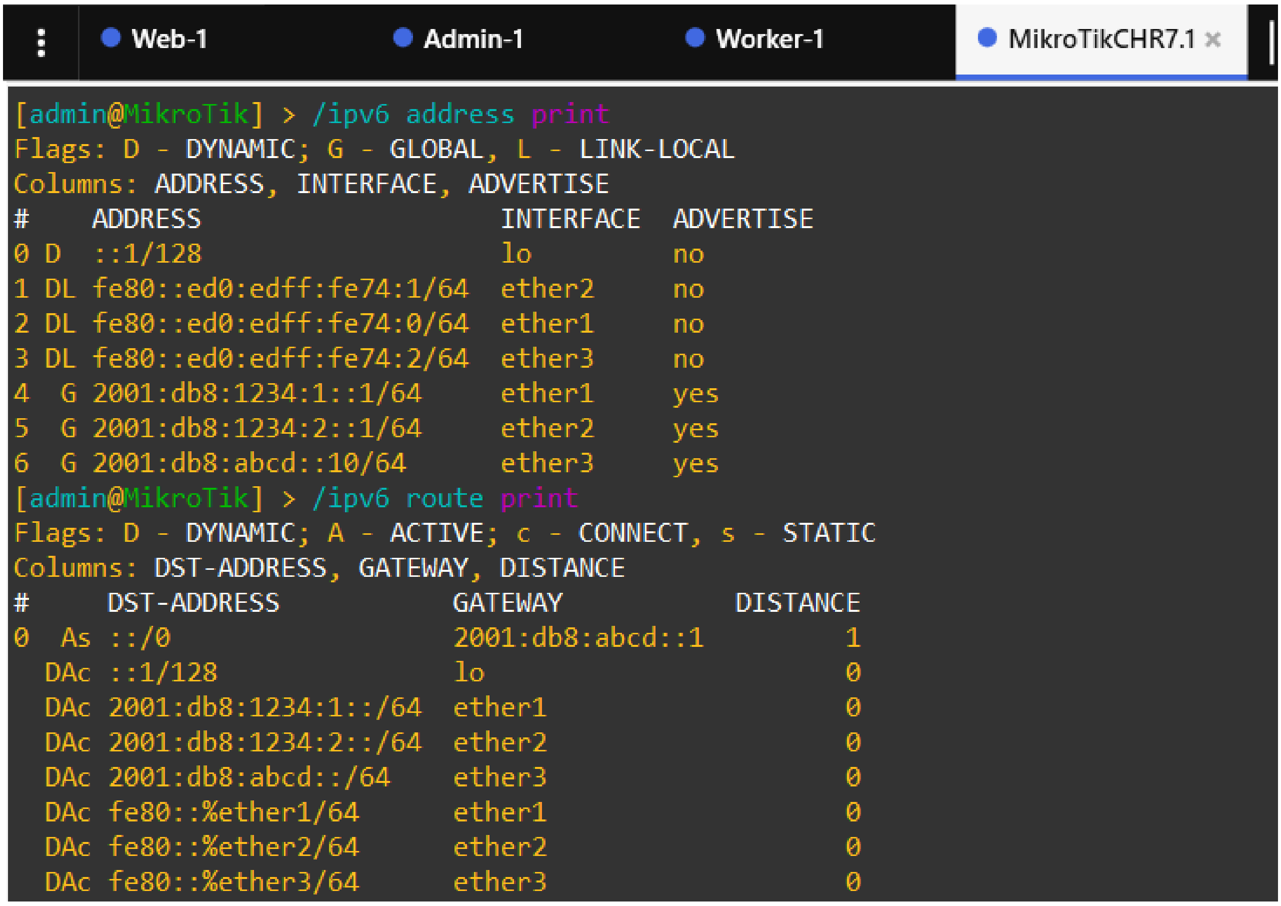


Figure 30 Netplan configuration on Worker




```
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:32:a1:cc:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.20.2/24 brd 192.168.20.255 scope global ens3
       valid_lft forever preferred_lft forever
    inet6 2001:db8:1234:2::2/64 scope global
       valid_lft forever preferred_lft forever
    inet6 fe80::e32:a1ff:fecc:0/64 scope link
       valid_lft forever preferred_lft forever
ubuntu@ubuntu-cloud:~$ ip -6 r
::1 dev lo proto kernel metric 256 pref medium
2001:db8:1234:2::/64 dev ens3 proto kernel metric 256 pref medium
fe80::/64 dev ens3 proto kernel metric 256 pref medium
default via 2001:db8:1234:2::1 dev ens3 proto static metric 1024 onlink pref medium
ubuntu@ubuntu-cloud:~$
```

Figure 31 Applied configuration on Worker




```
[admin@MikroTik] > /ipv6 address print
Flags: D - DYNAMIC; G - GLOBAL, L - LINK-LOCAL
Columns: ADDRESS, INTERFACE, ADVERTISE
#    ADDRESS                   INTERFACE  ADVERTISE
0 D  ::1/128                   lo         no
1 DL fe80::ed0:edff:fe74:1/64  ether2     no
2 DL fe80::ed0:edff:fe74:0/64  ether1     no
3 DL fe80::ed0:edff:fe74:2/64  ether3     no
4  G 2001:db8:1234:1::1/64     ether1     yes
5  G 2001:db8:1234:2::1/64     ether2     yes
6  G 2001:db8:abcd::10/64      ether3     yes
[admin@MikroTik] > /ipv6 route print
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT, s - STATIC
Columns: DST-ADDRESS, GATEWAY, DISTANCE
#      DST-ADDRESS           GATEWAY            DISTANCE
0  As  ::/0                  2001:db8:abcd::1          1
  DAc  ::1/128               lo                        0
  DAc  2001:db8:1234:1::/64  ether1                    0
  DAc  2001:db8:1234:2::/64  ether2                    0
  DAc  2001:db8:abcd::/64    ether3                    0
  DAc  fe80::%ether1/64      ether1                    0
  DAc  fe80::%ether2/64      ether2                    0
  DAc  fe80::%ether3/64      ether3                    0
```

Figure 32 Configured addresses and routes on Gateway

Then check connectivity between machines:

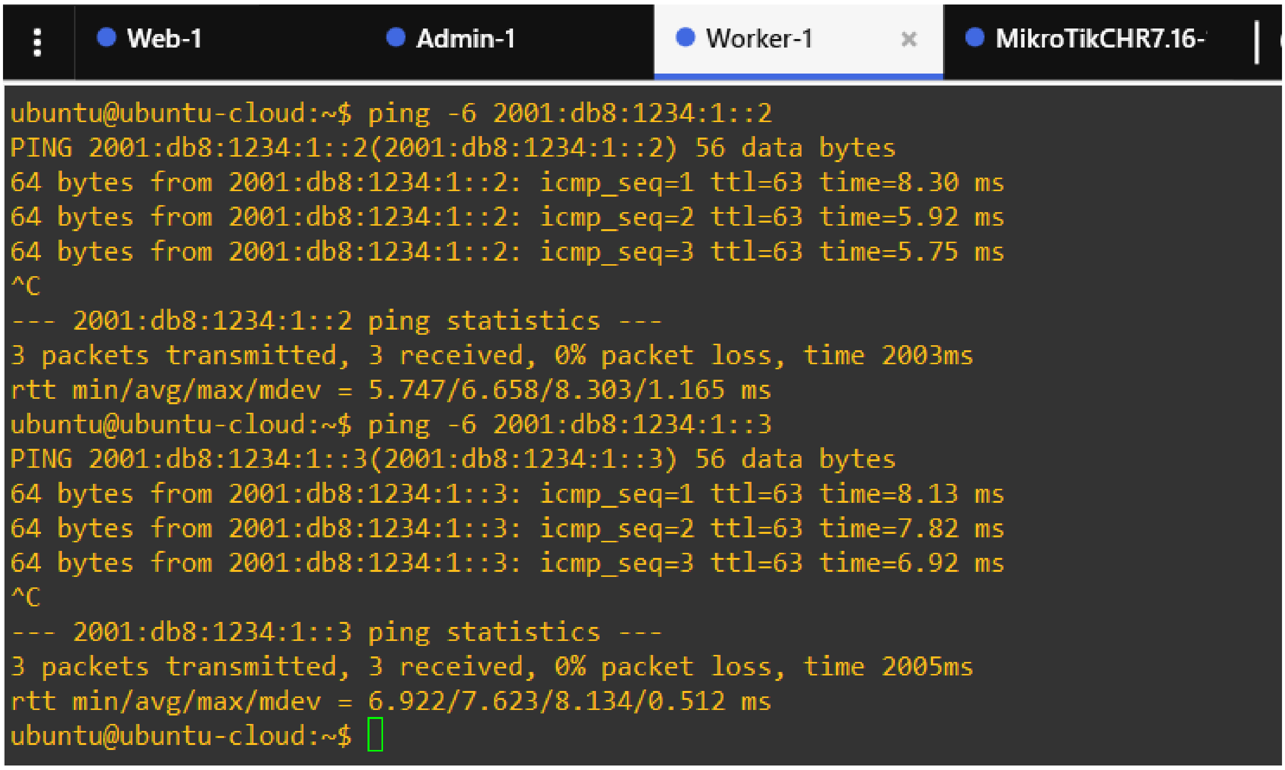


Figure 33 Checking IPv6 connectivity

2. Start capturing traffic on the link between Admin and Switch1:

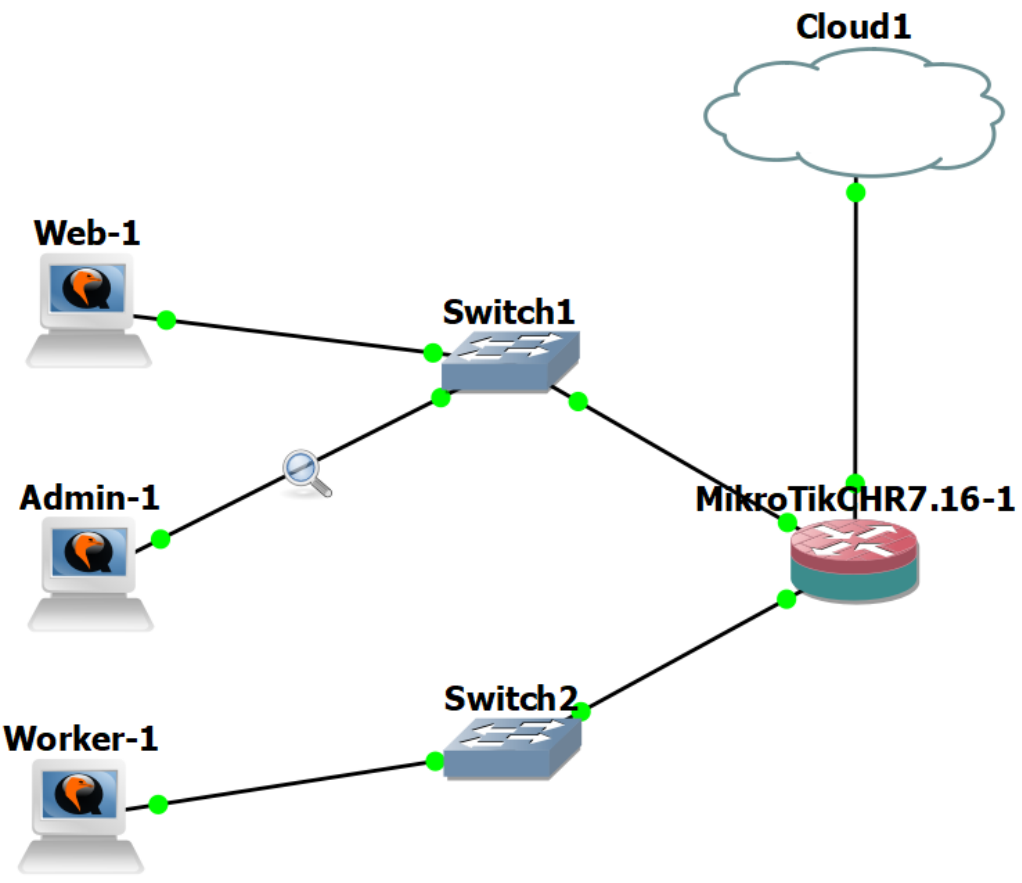


Figure 34 Capturing traffic on Admin-Switch1 link

Next accessing Web's http server using it's IPv6 address:

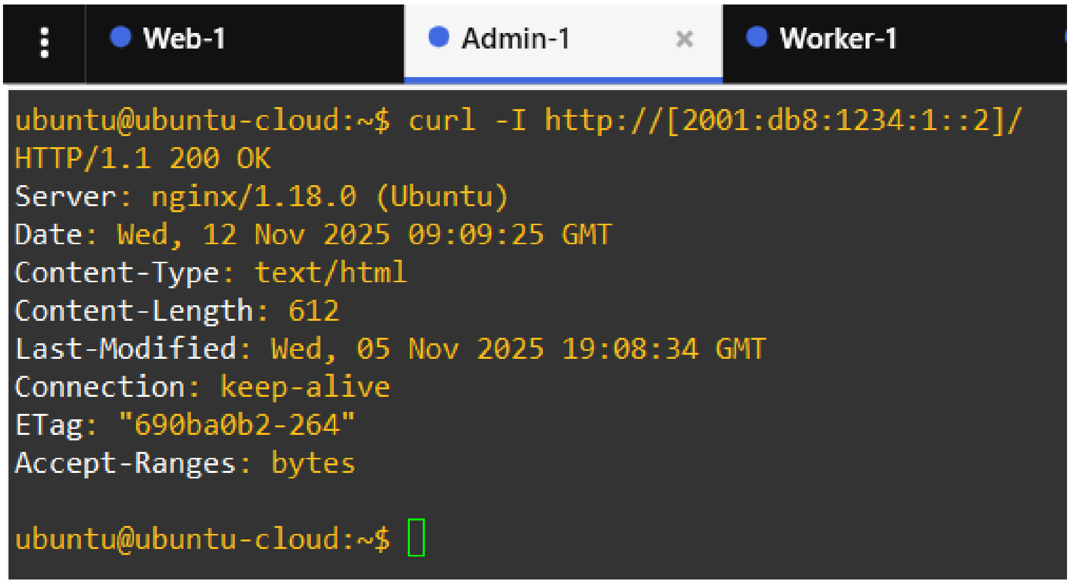


Figure 35 HTTP request to Web via curl

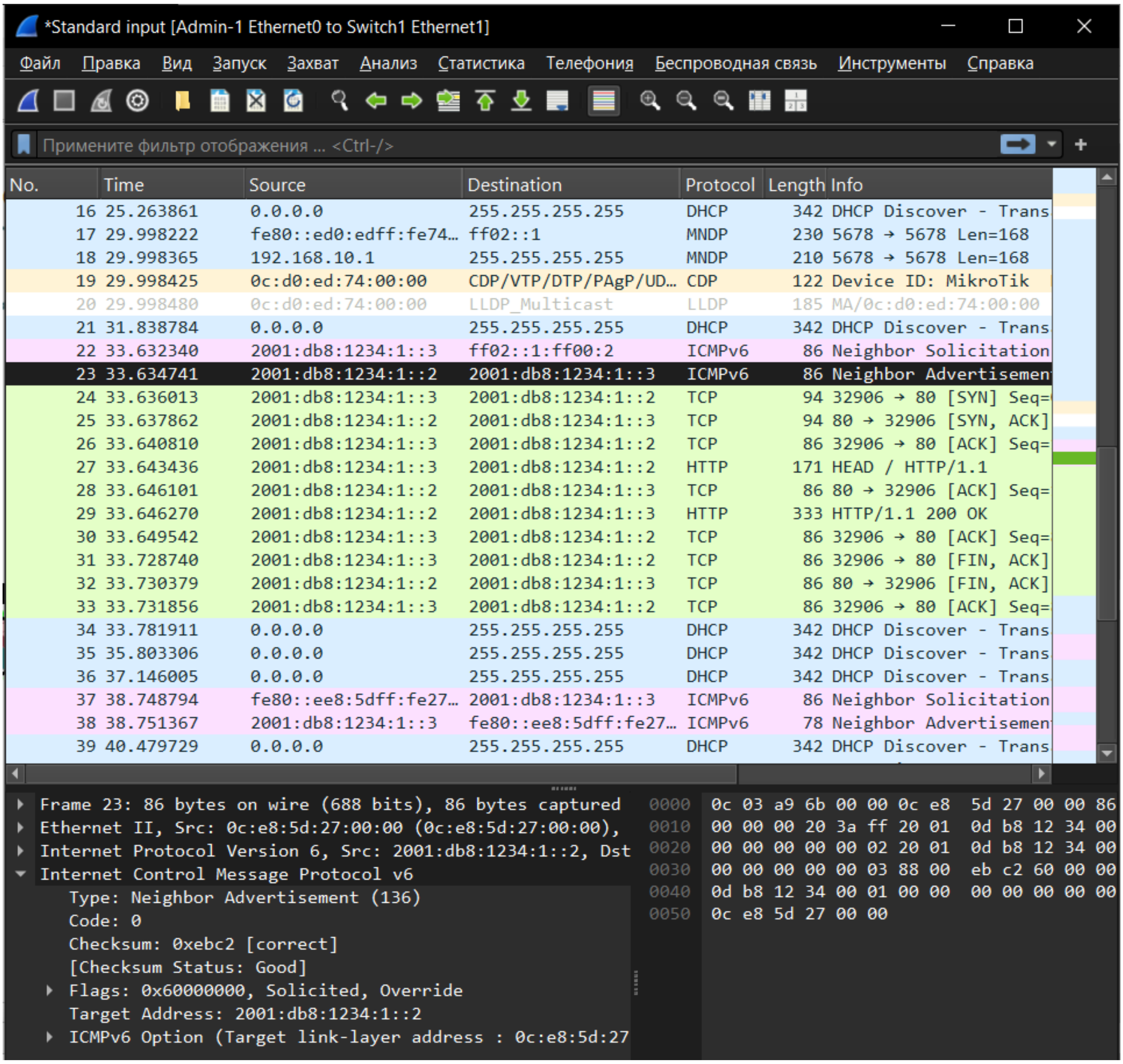


Figure 36 Captured HTTP IPv6 traffic

The main difference between IPv6 and IPv4 is with packets on Network layer. For IPv4 we have variable IP header size, from 20-60 bytes. For IPv6 we have fixed IP header size - 40 bytes.

The format of communicating remains almost the same, there's no changes in TCP or HTTP protocol layers.

We can also notice that instead of ARP requests, the ICMPv6 NDP neighbor solicitation requests are used to translate IP address to MAC address.

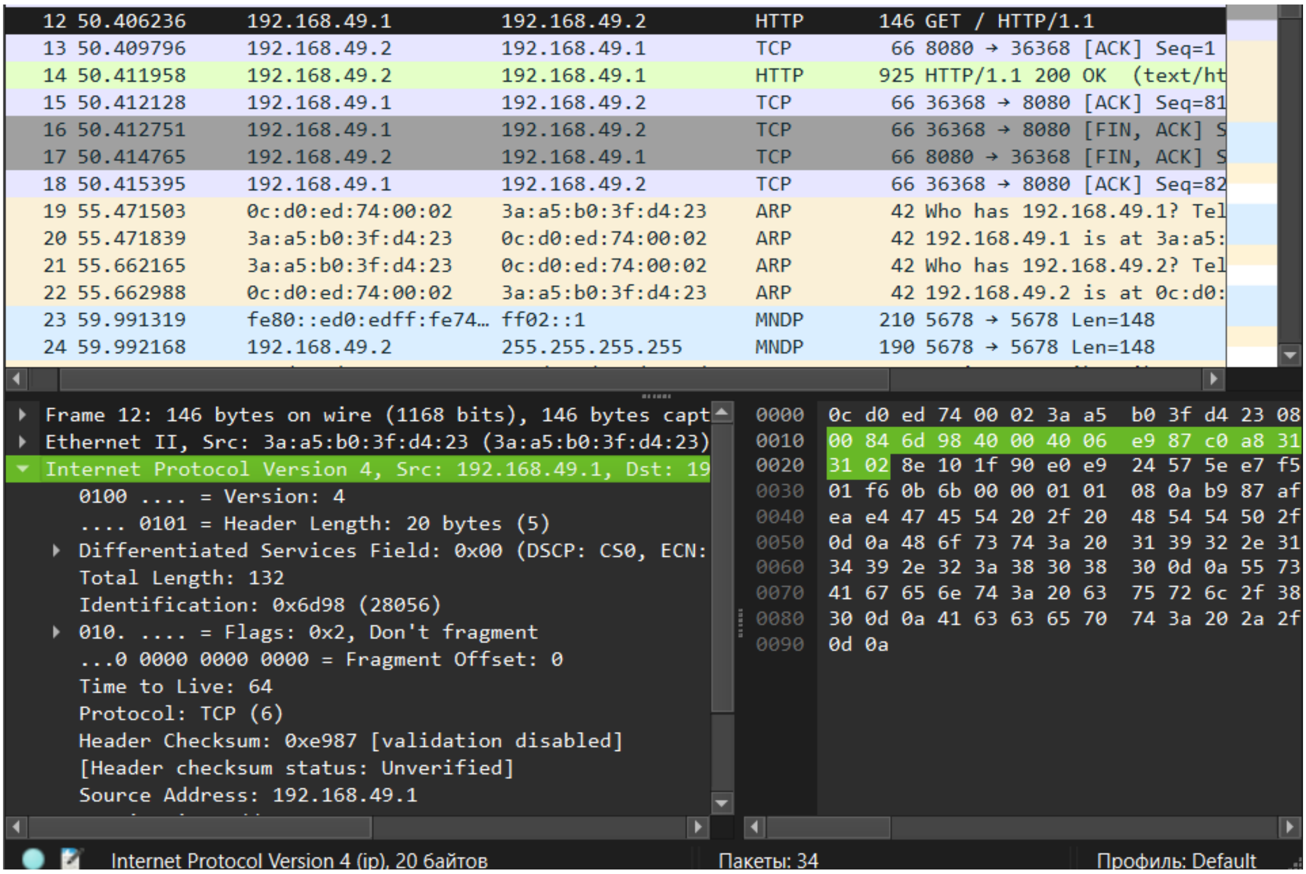


Figure 37 IP header in IPv4 HTTP request

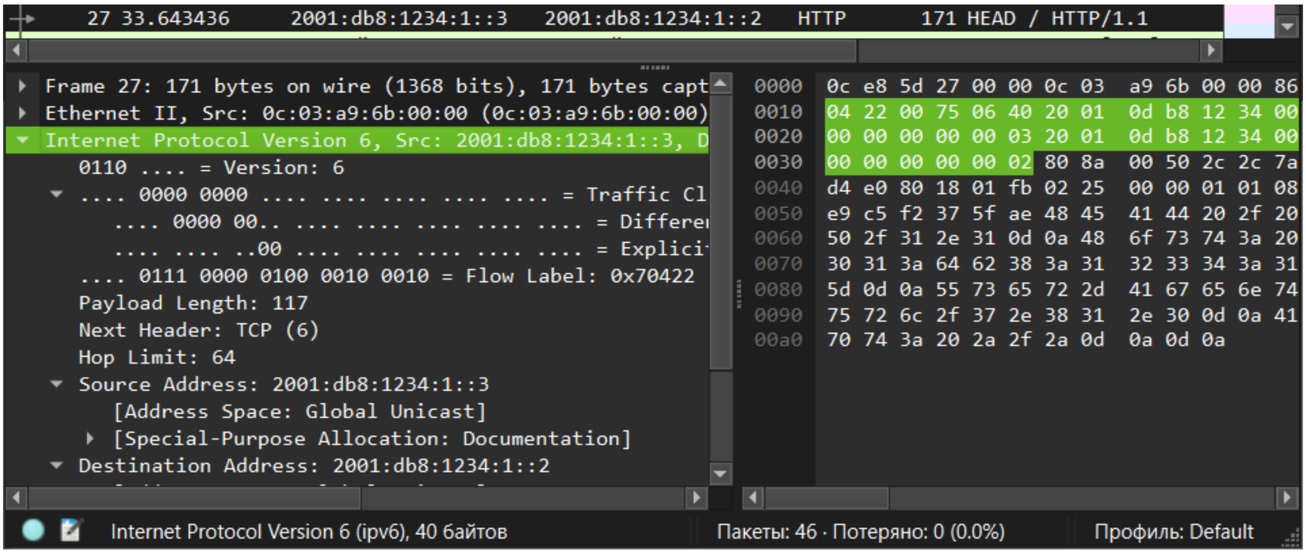


Figure 38 IP header in IPv6 HTTP request

3. For practice I will take Web and Worker IPv6 addressed:

   2001:db8:1234:1::2/64 and 2001:db8:1234:2::2/64

Web’s compressed address: 2001:db8:1234:1::2

First, we decompressing “::”, this represents the group of consecutive zero hex digits. This can be used only once in address and we need to get 8 groups of 4 hex digits, now we have 5 groups, so we need to add three groups of zeros: 2001:db8:1234:1:0000:0000:0000:2.

Next step is to prepend zeros in each group less than 4 digits. in our example this are groups 2, 4 and 8, prepending zeros there: 2001:0db8:1234:0001:0000:0000:0000:0002.

Web’s decompressed address: 2001:0db8:1234:0001:0000:0000:0000:0002

In this address octet “0001” represents subnet ID, and octet “0002” represents host ID in this subnet. /64 is subnet prefix.

Worker’s compressed address: 2001:db8:1234:2::2

For Worker’s address we doing completely the same steps. Decompressing chian of zeros:

2001:db8:1234:2:0000:0000:0000:2.

Then prepending zeros in each group to get 4 digits in each group:

2001:0db8:1234:0002:0000:0000:0000:0002.

Worker’s decompressed address: 2001:0db8:1234:0002:0000:0000:0000:0002

References:

1. Burp Suite community edition. https://portswigger.net/burp/communitydownload
2. Wireshark. https://www.wireshark.org/docs/
3. HTTP. https://en.wikipedia.org/wiki/HTTP
4. Secure Shell. https://en.wikipedia.org/wiki/Secure_Shell
5. IPv6. https://en.wikipedia.org/wiki/IPv6
6. Nmap. https://nmap.org/